**UNIVERSIDADE FEDERAL DA BAHIA**
**INSTITUTO DE COMPUTAÇÃO**
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

# TRABALHO 01 — CRIPTOGRAFIA APLICADA: PROTEÇÃO, ASSINATURAS DIGITAIS E CERTIFICADOS

*Estudo de Caso: Auditoria de Contratos Digitais na Miguelito Segurança e Tecnologia Ltda.*

Relatório Técnico apresentado à disciplina **MATC99 — Segurança e Auditoria de Sistemas de Informação** do Instituto de Computação da Universidade Federal da Bahia (UFBA), como requisito parcial de avaliação prática.

**Aluno:** Wellington Miguel de Jesus Silva
**Empresa Auditada (Cenário):** Miguelito Segurança e Tecnologia Ltda.

---

## SUMÁRIO

**1. Introdução e Contextualização do Problema**
**2. Fundamentação Teórica**
2.1. Criptografia Simétrica vs. Criptografia Assimétrica
2.2. Funcionamento do Modo AES-GCM e Integridade Autenticada (AEAD)
2.3. Infraestrutura de Chaves Públicas (PKI), Certificados X.509 e Assinaturas RSA
**3. Metodologia e Ambiente Experimental**
**4. Experimentos da Parte I: Proteção e Integridade com AES-256-GCM**
**5. Experimentos da Parte II: Assinaturas Digitais e Cadeia de Certificação**
**6. Matriz de Decisão e Recomendações de Governança**
**7. Conclusão**
**8. Referências Bibliográficas**

Salvador — BA | Novembro de 2026

# 1. Introdução e Contextualização do Problema

A digitalização de processos de contratação entre empresas (B2B) substituiu o papel físico e o reconhecimento de firma em cartório por fluxos eletrônicos baseados em rede. Embora essa transição reduza custos operacionais e acelere o fechamento de negócios, ela transfere toda a responsabilidade jurídica e financeira da transação para os controles criptográficos adotados pelas partes. Quando esses controles são mal projetados ou validados apenas de forma parcial, a organização fica exposta a fraudes financeiras e à perda de validade jurídica dos seus contratos.

Neste trabalho, analisamos o cenário da empresa fictícia **Miguelito Segurança e Tecnologia Ltda.** (CNPJ 14.382.910/0001-55), sediada em Salvador/BA, que atua na operação de sistemas críticos e contrata serviços de nuvem híbrida, links redundantes MPLS/IPSec e custódia de módulos HSM junto a fornecedores externos. Durante uma revisão de auditoria sobre a tramitação do **Contrato Comercial nº 2026/00849-MST**, celebrado com a fornecedora *Parceiro Infraestrutura e Cloud Computing Ltda.* (parceiro.miguelito.com.br, CNPJ 67.419.002/0001-83) no valor global de R$ 1.450.000,00 (dividido em 12 parcelas mensais de R$ 120.833,33), surgiram indícios de duas falhas graves no processo de troca e validação dos arquivos digitais: a suspeita de manipulação de cláusulas financeiras durante o trânsito do documento e o recebimento de contratos assinados com certificados digitais não reconhecidos pela governança interna.

Do ponto de vista de segurança da informação e perícia forense, o incidente enfrentado pela Miguelito Segurança e Tecnologia envolve três riscos práticos distintos que precisam ser isolados e tratados:

- **1. Interceptação e Modificação em Trânsito (*Man-in-the-Middle* / *Bit-Flipping*):** Contratos comerciais contêm dados bancários, valores de SLA, multas contratuais e especificações técnicas que não podem vazar para terceiros em trânsito (quebra de confidencialidade). Além disso, caso a empresa utilize um algoritmo de cifragem tradicional sem autenticação integrada, um atacante posicionado na rede pode alterar bytes específicos do texto cifrado para modificar valores no texto decifrado sem precisar conhecer a chave secreta.
- **2. Adulteração Documental Pós-Assinatura:** Mesmo quando um contrato é assinado por um representante legítimo, existe o risco de o arquivo sofrer modificações posteriores — seja por ação maliciosa interna, seja por interceptação — alterando, por exemplo, o valor da parcela mensal de R$ 120.833,33 para R$ 920.833,33. O mecanismo de auditoria precisa detectar alterações mínimas de um único caractere e rejeitar o documento imediatamente.
- **3. Falsificação de Identidade Criptográfica (*Key Spoofing via Rogue CA*):** Um invasor externo pode gerar seu próprio par de chaves assimétricas, configurar uma Autoridade Certificadora própria (*Rogue CA*) e emitir um certificado digital preenchido com os mesmos dados cadastrais do fornecedor verdadeiro (CN = parceiro.miguelito.com.br). Se o setor responsável pela conferência verificar apenas se a assinatura digital é matematicamente válida contra a chave pública anexada ao processo — esquecendo de verificar quem emitiu aquele certificado —, o contrato fraudulento será aceito como autêntico.

Para responder a esse cenário, este trabalho implementa e documenta uma bancada experimental dividida em duas frentes complementares: na **Parte I**, utilizamos a biblioteca cryptography do Python para proteger o contrato com a cifra simétrica autenticada **AES-256-GCM**; na **Parte II**, utilizamos o **OpenSSL** em ambiente Linux para construir uma Infraestrutura de Chaves Públicas (PKI) completa, confrontando a Autoridade Certificadora interna da empresa (Miguelito Root CA) com uma autoridade atacante (Rogue CA) ao longo de três cenários de auditoria (Cenários A, B e C).

# 2. Fundamentação Teórica

## 2.1. Criptografia Simétrica vs. Criptografia Assimétrica

A escolha entre algoritmos simétricos e assimétricos em uma arquitetura corporativa decorre diretamente das diferenças de desempenho, gestão de chaves e propriedades jurídicas de cada modelo. Na **criptografia simétrica**, o emissor e o destinatário compartilham exatamente a mesma chave secreta $K$, usada tanto para cifrar quanto para decifrar os dados. Algoritmos simétricos modernos, como o AES (*Advanced Encryption Standard*), operam sobre blocos de bits por meio de rodadas sucessivas de substituição, permutação e combinação linear com subchaves derivadas. Por serem operações leves e contarem com instruções dedicadas diretamente no hardware dos processadores atuais (*AES-NI*), as cifras simétricas atingem altíssimo desempenho, sendo a escolha ideal para cifrar arquivos extensos e tráfego de rede.

Por outro lado, a criptografia simétrica possui duas limitações estruturais para ambientes de negócios abertos: (i) o **problema da distribuição de chaves**, pois exige um canal seguro prévio para combinar a chave $K$ entre cada par de empresas (em uma rede com $n$ participantes, seriam necessárias $n(n-1)/2$ chaves distintas); e (ii) a **ausência de**

**não-repúdio (irretratabilidade)**. Como tanto a contratante quanto a contratada possuem a mesma chave secreta *K*, ambas têm capacidade técnica idêntica para gerar um arquivo cifrado e sua tag de verificação. Em uma disputa judicial, a Miguelito não conseguiria provar a um perito externo que determinado arquivo veio exclusivamente do fornecedor, pois a própria Miguelito poderia tê-lo produzido.

A **criptografia assimétrica (ou de chave pública)** resolve esses dois limites ao substituir a chave compartilhada por um par de chaves matematicamente vinculadas: uma **chave privada** (mantida sob custódia exclusiva do titular) e uma **chave pública** (divulgada abertamente). Sua segurança baseia-se em problemas matemáticos difíceis de inverter sem o segredo privado, como a fatoração do produto de dois números primos grandes no algoritmo RSA. Embora seja computacionalmente mais pesada que a simétrica, a criptografia de chave pública viabiliza as **assinaturas digitais**: o titular assina o resumo do documento com sua chave privada e qualquer auditor verifica a assinatura com a chave pública. Como apenas o titular possui a chave privada, ele não pode negar a autoria posteriormente, garantindo o não-repúdio jurídico.

### 2.2. Funcionamento do Modo AES-GCM e Integridade Autenticada (AEAD)

Durante muitos anos, sistemas corporativos utilizaram cifras de bloco em modos de operação voltados apenas para confidencialidade, como o CBC (*Cipher Block Chaining*) ou o CTR (*Counter Mode*). Embora esses modos escondam o conteúdo contra observadores passivos, eles não protegem a integridade contra atacantes ativos. No modo Contador (CTR) puro, por exemplo, o algoritmo recebe um valor único de 96 bits (12 bytes) chamado **Nonce** (IV), concatena um contador sequencial de 32 bits (bloco inicial $J_0$) e cifra esse contador com a chave *K* para gerar um fluxo pseudoaleatório (*keystream*), que é combinado via operação lógica **XOR** com cada bloco de texto em claro $P_i$ para gerar o bloco cifrado $C_i$.

O risco de usar o modo CTR sozinho está na sua **maleabilidade linear**: como decifrar consiste apenas em aplicar novamente o XOR entre o texto cifrado e o mesmo *keystream*, se um invasor interceptar o arquivo e inverter 1 bit em uma posição conhecida do bloco cifrado $C_i$, a decifragem ocorrerá sem erro algum e inverterá exatamente o mesmo bit no texto recuperado $P_i$ (ataque de *bit-flipping*). Para resolver isso de forma nativa, o padrão **NIST SP 800-38D** especificou o **AES-GCM** (*Galois/Counter Mode*), um esquema **AEAD** (*Authenticated Encryption with Associated Data*) que acopla o modo contador à função de autenticação algébrica **GHASH**:

- **1. Subchave de Autenticação e Dados Associados (AAD):** Cifra-se um bloco nulo de 128 bits com a chave *K* para derivar a subchave interna $H = E_K(0^{128})$. Além do texto cifrado, o GCM permite autenticar metadados em texto claro (como identificação do contrato), chamados de AAD.
- **2. Polinômio GHASH em GF($2^{128}$) e Authentication Tag:** Os blocos de 128 bits do AAD e os blocos já cifrados ($C_1$, ..., $C_n$) entram como coeficientes de um polinômio avaliado no ponto secreto *H* dentro do corpo finito de Galois GF($2^{128}$), definido pelo polinômio irredutível $P(x) = x^{128} + x^7 + x^2 + x + 1$. O resultado do GHASH sofre um XOR com o contador inicial cifrado $E_K(J_0)$, formando a **Authentication Tag (T)** de 16 bytes (128 bits):

$$T = GHASH_H(AAD, Ciphertext) \oplus E_K(J_0)$$

Na decifragem (arquitetura *Encrypt-then-MAC*), o destinatário recalcula a tag local *T'* a partir do Nonce, AAD e Ciphertext recebidos antes de liberar qualquer dado. Se um atacante alterar um byte do bloco $C_i$ por uma diferença não nula Δ, a diferença entre a tag recalculada e a original em GF($2^{128}$) será $\Delta T = \Delta \cdot H^{n-i+2} \pmod{P(x)}$. Como um corpo finito não possui divisores de zero, o produto de dois elementos não nulos nunca é zero ($T' \neq T$). O algoritmo detecta a divergência com certeza matemática e aborta a operação lançando a exceção InvalidTag.

### 2.3. Infraestrutura de Chaves Públicas (PKI), Certificados X.509 e Assinaturas RSA

Para garantir autenticidade de origem e não-repúdio jurídico entre empresas distintas, utiliza-se a assinatura digital **RSA com SHA-256 (RFC 8017 — PKCS#1)** no esquema *Hash-then-Sign*. Primeiro, o documento passa pela função hash **SHA-256**, que gera um resumo único de 32 bytes (256 bits) dotado de forte **efeito avalanche** (qualquer alteração mínima no texto muda cerca de 50% dos bits do hash) e resistência a colisões. No padrão **PKCS#1 v1.5** (ou de forma probabilística com salt no padrão **RSA-PSS**), esse resumo e o identificador do algoritmo SHA-256 são estruturados em um bloco ASN.1 preenchido até o tamanho da chave (256 bytes para RSA 2048 bits), chamado de *EM*. O signatário aplica sua chave privada *(d, N)* por exponenciação modular ($S = EM^d \bmod N$) para gerar a assinatura; quem verifica aplica a chave pública *(e, N)* sobre a assinatura ($EM' = S^e \bmod N$) e compara o hash extraído com o hash do arquivo.

Entretanto, a verificação matemática isolada prova apenas que o documento foi assinado pelo par daquela chave pública, mas não informa a quem essa chave pertence no mundo real. É a **Infraestrutura de Chaves Públicas (PKI / X.509 v3 — RFC 5280)** que resolve esse problema: uma **Autoridade Certificadora Raiz (Root CA)** confiável —

no nosso estudo, a Miguelito Root CA, instalada na *Trust Store* da empresa — verifica a identidade do fornecedor e assina digitalmente o seu certificado X.509 (parceiro.crt), amarrando o nome do titular (Subject: CN = parceiro.miguelito.com.br) à sua chave pública e fixando restrições de uso (basicConstraints: CA:FALSE e keyUsage: digitalSignature, nonRepudiation). Assim, uma auditoria completa exige sempre duas etapas: validar matematicamente a assinatura do documento e validar a cadeia do certificado até a Root CA corporativa, consultando ainda sua não-revogação (via CRL ou OCSP).

## 3. Metodologia e Ambiente Experimental

Os experimentos foram conduzidos em ambiente Linux (Ubuntu 24.04 LTS sob WSL2), utilizando scripts idempotentes em Python e GNU Bash para assegurar total reprodutibilidade e registro forense dos resultados. A Tabela 1 apresenta a especificação das ferramentas empregadas.

| Componente / Ferramenta | Versão / Especificação | Finalidade nos Experimentos |
|---|---|---|
| Sistema Operacional | Ubuntu 24.04 LTS (Kernel 6.6.87.2 WSL2) | Ambiente POSIX isolado para execução dos scripts e coleta de logs. |
| Interpretador Python | Python 3.12.3 / 3.13.14 | Execução do módulo parte1_aes_gcm.py com logging e type hints. |
| Biblioteca Criptográfica | cryptography 41.0.7 / 50.0.2 (AESGCM) | Cifragem autenticada AES-256-GCM e captura de InvalidTag. |
| Suíte OpenSSL CLI | OpenSSL 3.0.13 (30 Jan 2024) | Geração de chaves RSA 4096/2048-bit, emissão X.509 v3, dgst e verify. |
| Contrato Comercial Base | contrato_miguelito.txt (2.841 bytes) | Contrato nº 2026/00849-MST (R$ 1.450.000,00 / 12x R$ 120.833,33). |

## 4. Experimentos da Parte I: Proteção e Integridade com AES-256-GCM

### 4.1. Procedimentos de Cifragem, Empacotamento e Decifragem

O script parte1_aes_gcm.py inicia carregando o arquivo contrato_miguelito.txt (2.841 bytes) e registrando seu hash SHA-256 original (9147c03987856361...ffe0cfe67). Em seguida, gera uma chave simétrica aleatória de 256 bits (32 bytes) via AESGCM.generate_key(256), um Nonce de 96 bits (12 bytes) via os.urandom(12) e define os Dados Associados (AAD) como MATC99-UFBA|MIGUELITO-SEGURANCA-E-TECNOLOGIA|CONTRATO-2026-00849. O arquivo resultante contrato_miguelito.enc é gravado em disco com exatos **2.869 bytes**, estruturados na forma: **Nonce (12B) || Ciphertext (2.841B) || Authentication Tag (16B)**. Na sequência, o script lê o arquivo do disco, valida a Tag de 128 bits e recupera contrato_miguelito_recuperado.txt, confirmando igualdade estrita byte a byte (original == recuperado: True) e mesmo resumo SHA-256.

### 4.2. Logs de Execução e Análise dos Cenários de Falha

Após validar a decifragem legítima, o script executou os dois cenários de falha exigidos. O quadro abaixo reproduz o log real capturado em parte1_aes_gcm.log:

```
[2026-11-04 15:36:36.889] [INFO] === Auditoria Criptográfica - Miguelito Segurança e Tecnologia (Parte I: AES-256-GCM) ===
[2026-11-04 15:36:36.894] [INFO] Contrato carregado: contrato_miguelito.txt (2841 bytes)
[2026-11-04 15:36:36.895] [INFO] SHA-256 original: 9147c03987856361e26a46f794f9588fe0850329cc2eaa28d5f0fd6ffe0cfe67
[2026-11-04 15:36:36.901] [INFO] Arquivo cifrado salvo em: contrato_miguelito.enc (2869 bytes totais)
[2026-11-04 15:36:36.901] [INFO]   Chave AES-256 (32B) : 185c3c58dc5153a9b84ddaf0b85ec9933e52f67c44f8d150f42151a460aa8a28
[2026-11-04 15:36:36.902] [INFO]   Nonce / IV (12B)    : 1d5d2cb6580417d1f29dbf07
[2026-11-04 15:36:36.902] [INFO]   AAD                 : MATC99-UFBA|MIGUELITO-SEGURANCA-E-TECNOLOGIA|CONTRATO-2026-00849
[2026-11-04 15:36:36.903] [INFO]   Auth Tag (16B)      : f987ee9b9f25a4ea8cbcac904a675b94
[2026-11-04 15:36:36.909] [INFO] Comparação de bytes (original == recuperado): True (2841 bytes)
[2026-11-04 15:36:36.910] [INFO] --- [Teste de Falha 1: Adulteração de 1 byte no Ciphertext (Bit-Flipping)] ---
[2026-11-04 15:36:36.910] [INFO] Byte modificado no offset 1420 (0x058C): 0x4B -> 0x4A
[2026-11-04 15:36:36.910] [WARNING] Exceção capturada com sucesso: cryptography.exceptions.InvalidTag
[2026-11-04 15:36:36.911] [INFO] Motivo: A alteração no bloco cifrado mudou o resultado do polinômio GHASH em GF(2^128).
[2026-11-04 15:36:36.911] [INFO] --- [Teste de Falha 2: Tentativa de Decifragem com Chave Incorreta] ---
[2026-11-04 15:36:36.911] [INFO] Chave correta : 185c3c58dc5153a9b84ddaf0b85ec9933e52f67c44f8d150f42151a460aa8a28
[2026-11-04 15:36:36.912] [INFO] Chave errada  : 8908f8e494c7d2089963f9ca67836a1d43bfc0a0ea511d3343b5c0a40d8df58e
[2026-11-04 15:36:36.912] [WARNING] Exceção capturada com sucesso: cryptography.exceptions.InvalidTag
```

No **Teste de Falha 1 (Bit-Flipping)**, invertemos apenas 1 bit no byte central do texto cifrado (offset 1420 / 0x058C, mudando 0x4B para 0x4A via XOR 0x01). Mesmo com a chave correta, o polinômio GHASH em GF($2^{128}$) produziu uma tag divergente de f987ee9b..., lançando cryptography.exceptions.InvalidTag e bloqueando a saída. No **Teste de Falha 2 (Chave Espúria)**, a tentativa de decifrar o arquivo íntegro com a chave incorreta 8908f8e4... alterou completamente a subchave *H′* e a máscara do contador inicial, resultando novamente no bloqueio imediato via InvalidTag. Ficam assim comprovadas as propriedades de **confidencialidade**, **integridade do criptograma** e **autenticidade simétrica**.

## 5. Experimentos da Parte II: Assinaturas Digitais e Cadeia de Certificação

### 5.1. Estrutura da PKI Simulada (CA Corporativa vs. Rogue CA)

O script parte2_pki_testes.sh estruturou o diretório pki_miguelito/ com duas hierarquias X.509 v3 distintas: (1) a **Cadeia Corporativa Legítima**, encabeçada pela autoridade raiz interna Miguelito Root CA (RSA 4096 bits, autoassinado, Serial 362E0342...2D96, CA:true), que assinou o certificado legítimo do fornecedor parceiro.crt (CN = parceiro.miguelito.com.br, RSA 2048 bits, Serial 21CFB1F0...BCC2, CA:FALSE); e (2) a **Cadeia Atacante (Rogue CA)**, composta por uma autoridade externa não confiável (Rogue Root CA, RSA 4096 bits) que emitiu o certificado falso parceiro_forjado.crt copiando o mesmo CN = parceiro.miguelito.com.br do fornecedor.

### 5.2. Análise do Cenário A: Documento Íntegro e Cadeia Válida

No Cenário A, o contrato original foi assinado com a chave privada legítima do fornecedor e submetido à verificação matemática da assinatura e à validação da cadeia contra a Miguelito Root CA:

```
# Comandos executados no Cenário A:
$ openssl dgst -sha256 -sign pki_miguelito/parceiro_legitimo/parceiro.key -out cenario_a.sig contrato_miguelito.txt
$ openssl dgst -sha256 -verify pki_miguelito/parceiro_legitimo/parceiro_pub.pem -signature cenario_a.sig contrato_miguelito.txt
$ openssl verify -CAfile pki_miguelito/root_ca/miguelito_root_ca.crt pki_miguelito/parceiro_legitimo/parceiro.crt

# Saída capturada em parte2_pki_testes.log:
[2026-11-04 15:36:40.697] [INFO] Assinatura gerada: cenario_a.sig (256 bytes)
[2026-11-04 15:36:40.721] [INFO] Verificação da assinatura (openssl dgst -verify): Verified OK
[2026-11-04 15:36:40.743] [INFO] Validação da cadeia contra Miguelito Root CA: pki_miguelito/parceiro_legitimo/parceiro.crt: OK
[2026-11-04 15:36:40.750] [INFO] Resultado Cenário A -> DECISÃO: ACEITAR
```

**Análise e Justificativa (DECISÃO: ACEITAR):** O retorno Verified OK comprova que o hash SHA-256 recuperado da assinatura cenario_a.sig (256 bytes) coincide exatamente com o hash de contrato_miguelito.txt. Simultaneamente, parceiro.crt: OK confirma que o certificado do signatário foi emitido pela Miguelito Root CA. Estão atendidos os critérios de integridade, autenticidade de identidade e não-repúdio.

### 5.3. Análise do Cenário B: Adulteração de Conteúdo Pós-Assinatura

No Cenário B, simulou-se uma fraude financeira alterando 1 byte no contrato após a assinatura (mudança da parcela mensal de R$ 120.833,33 para R$ 920.833,33 via sed), verificando-o em seguida contra a assinatura legítima colhida no Cenário A:

```
# Comandos executados no Cenário B:
$ sed 's/R\$ 120\.833,33/R\$ 920.833,33/g' contrato_miguelito.txt > contrato_adulterado.txt
$ openssl dgst -sha256 -verify pki_miguelito/parceiro_legitimo/parceiro_pub.pem -signature cenario_a.sig contrato_adulterado.txt

# Saída capturada em parte2_pki_testes.log:
[2026-11-04 15:36:40.824] [INFO] SHA-256 original   : 9147c03987856361e26a46f794f9588fe0850329cc2eaa28d5f0fd6ffe0cfe67
[2026-11-04 15:36:40.832] [INFO] SHA-256 adulterado : 817d7fae7346f151320d2469a8546c52edefa478219948e1efbb75edf9f0a8d2
[2026-11-04 15:36:40.856] [WARN] Código de saída do OpenSSL: 1
  40C71E4E407F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:430:
  Verification failure
[2026-11-04 15:36:40.868] [WARN] Resultado Cenário B -> DECISÃO: REJEITAR
```

**Análise e Justificativa (DECISÃO: REJEITAR):** A troca do dígito '1' por '9' acionou o efeito avalanche do SHA-256, alterando completamente o resumo de 256 bits (de 9147c039... para 817d7fae...). Ao abrir o bloco PKCS#1 v1.5 da assinatura com a chave pública legítima, a rotina ossl_rsa_verify constatou a divergência entre o hash assinado e o hash do arquivo modificado, retornando bad signature / Verification failure (Exit Code 1).

### 5.4. Análise do Cenário C: Certificado Emitido por Autoridade Não Confiável

No Cenário C, o contrato original íntegro foi assinado com a chave privada falsa (parceiro_forjado.key) vinculada ao certificado emitido pela Rogue CA:

```
# Comandos executados no Cenário C:
$ openssl dgst -sha256 -sign pki_miguelito/parceiro_forjado/parceiro_forjado.key -out cenario_c_forjado.sig contrato_miguelito.txt
$ openssl dgst -sha256 -verify pki_miguelito/parceiro_forjado/parceiro_forjado_pub.pem -signature cenario_c_forjado.sig contrato_mi
$ openssl verify -CAfile pki_miguelito/root_ca/miguelito_root_ca.crt pki_miguelito/parceiro_forjado/parceiro_forjado.crt

# Saída capturada em parte2_pki_testes.log:
[2026-11-04 15:36:40.927] [INFO] 1) Verificação matemática com a chave pública do certificado falso: Verified OK
[2026-11-04 15:36:40.949] [WARN] 2) Validação da cadeia contra a Trust Store da Miguelito (Exit Code 2):
  C = BR, ST = Sao Paulo, L = Sao Paulo, O = Parceiro Infraestrutura e Cloud Computing Ltda, OU = Contratos Digitais B2B, CN = parc
  error 20 at 0 depth lookup: unable to get local issuer certificate
  error pki_miguelito/parceiro_forjado/parceiro_forjado.crt: verification failed
[2026-11-04 15:36:40.962] [WARN] Resultado Cenário C -> DECISÃO: REJEITAR
```

**Diferenciação entre Integridade e Autenticidade (DECISÃO: REJEITAR):** Embora a verificação matemática isolada retorne Verified OK (pois o documento está íntegro em relação ao par de chaves gerado pelo próprio atacante), a validação de cadeia falha imediatamente no nível 0 (at 0 depth lookup) com o erro error 20: unable to get local issuer certificate. Isso ocorre porque o emissor do certificado (Rogue Root CA) não consta na *Trust Store* da Miguelito Segurança e Tecnologia, caracterizando tentativa de falsificação ideológica (*spoofing* de certificado).

## 6. Matriz de Decisão e Recomendações de Governança

### 6.1. Matriz Comparativa de Auditoria

| Cenário Auditado | Integridade Matemática | Confiabilidade da Identidade / Chave | Evidência Técnica Capturada | Veredito |
|---|---|---|---|---|
| Parte I — Fluxo Normal | Válida (Tag GCM e SHA-256 OK) | Válida (Chave simétrica correta) | 2.841 bytes recuperados sem erro | **ACEITAR** |
| Parte I — Bit-Flipping | Inválida (GHASH diverge) | Válida (Chave simétrica correta) | cryptography.exceptions.InvalidTag | **REJEITAR** |
| Parte I — Chave Espúria | Não verificável (H' ≠ H) | Inválida (Chave não autorizada) | cryptography.exceptions.InvalidTag | **REJEITAR** |
| Parte II — Cenário A | Válida (Verified OK) | Válida (Miguelito Root CA: OK) | Exit Code 0 em dgst e verify | **ACEITAR** |
| Parte II — Cenário B | Inválida (Verification failure) | Válida (Certificado legítimo) | ossl_rsa_verify:bad signature (Exit 1) | **REJEITAR** |
| Parte II — Cenário C | Válida (Verified OK isolado) | Inválida (Fora da Trust Store) | error 20: unable to get local issuer (Exit 2) | **REJEITAR** |

### 6.2. Recomendações Técnicas para a Miguelito Segurança e Tecnologia

- **1. Validação Obrigatória em Duas Etapas e Fixação de Trust Store:** O pipeline de recebimento de contratos deve executar primeiro openssl verify -CAfile miguelito_root_ca.crt antes de conferir a assinatura RSA, bloqueando qualquer certificado emitido por CAs externas não homologadas.
- **2. Guarda da Root CA Offline em HSM e Uso de CA Intermediária:** Manter a chave privada RSA 4096 bits da Miguelito Root CA isolada em hardware (HSM FIPS 140-3) e criar uma Autoridade Certificadora Intermediária para assinar os certificados operacionais dos parceiros comerciais.
- **3. Verificação de Revogação (CRL / OCSP) e Extensões X.509 v3:** Publicar Listas de Certificados Revogados (CRL) e ativar consulta OCSP (-crl_check) para recusar chaves privadas comprometidas antes do vencimento, além de exigir CA:FALSE e keyUsage: digitalSignature, nonRepudiation.
- **4. Proibição de Reuso de Nonce e Rotação de Chaves no AES-GCM:** Garantir que cada operação no AES-256-GCM utilize um Nonce de 12 bytes único gerado por CSPRNG e rotacionar periodicamente as chaves simétricas, prevenindo a exposição do *keystream* e da subchave *H* do GHASH.

## 7. Conclusão

O trabalho experimental realizado para o cenário da **Miguelito Segurança e Tecnologia Ltda.** demonstrou na prática como a cifragem simétrica autenticada e a infraestrutura de chaves públicas atuam de forma complementar na proteção de contratos digitais. Na Parte I, comprovou-se que o modo **AES-256-GCM** elimina a maleabilidade do modo contador puro por meio do polinômio **GHASH** em GF($2^{128}$), bloqueando instantaneamente tanto a adulteração de 1 bit no texto cifrado quanto o uso de chaves espúrias através da exceção InvalidTag.

Na Parte II, os testes com o OpenSSL evidenciaram a distinção fundamental entre integridade matemática de dados e autenticidade da identidade do signatário. Enquanto o Cenário B mostrou que qualquer fraude pós-assinatura em valores contratuais é barrada pelo efeito avalanche do **SHA-256** (Verification failure), o Cenário C provou que uma assinatura matematicamente válida (Verified OK) deve ser rejeitada quando seu certificado não possui vínculo de confiança com a Miguelito Root CA (error 20: unable to get local issuer certificate). A adoção conjunta desses dois controles restabelece integralmente a confidencialidade, a integridade, a autenticidade e o não-repúdio nas operações da empresa.

## 8. Referências Bibliográficas

1. **DWORKIN, Morris J.** *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication (SP) 800-38D, National Institute of Standards and Technology, Gaithersburg, MD, Nov. 2007.
2. **COOPER, D.; SANTESSON, S.; FARRELL, S.; BOEYEN, S.; HOUSLEY, R.; POLK, W.** *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. IETF RFC 5280, Maio 2008.
3. **MORIARTY, K.; KALISKI, B.; JONSSON, J.; RUSCH, A.** *PKCS #1: RSA Cryptography Specifications Version 2.2*. IETF RFC 8017, Nov. 2016.
4. **NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST).** *Secure Hash Standard (SHS)*. FIPS PUB 180-4, U.S. Department of Commerce, Ago. 2015.
5. **STALLINGS, William.** *Criptografia e Segurança de Redes: Princípios e Práticas*. 8ª ed. São Paulo: Pearson Education, 2023.
6. **PYTHON CRYPTOGRAPHIC AUTHORITY (PyCA).** *Cryptography Documentation — Authenticated Encryption with Associated Data (AESGCM)*. Disponível em: https://cryptography.io/en/latest/hazmat/primitives/aead/.